

C++ Fundamentals to OOP

PART 1: FUNDAMENTALS

1. Introduction to C++

C++ is a general-purpose, compiled programming language that extends C with object-oriented features. It gives you low-level control over memory (like C) while also supporting high-level abstractions like classes and objects.

Real-world example: Game engines (Unreal Engine), operating systems components, browsers (Chrome's rendering engine), and embedded systems all use C++ because it offers both speed and structure.

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```

- `#include <iostream>` — brings in input/output functionality
 - `main()` — the entry point every C++ program starts from
 - `cout` — used to print output
 - `return 0` — tells the OS the program ended successfully
-

2. Variables and Data Types

A variable is a named location in memory that stores a value of a specific type.

Type	Size (typical)	Example
int	4 bytes	int age = 20;
float	4 bytes	float price = 9.99f;
double	8 bytes	double pi = 3.14159;
char	1 byte	char grade = 'A';
bool	1 byte	bool isActive = true;
string	varies	string name = "Ahmad";

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int studentAge = 20;
    double gpa = 3.75;
    char grade = 'A';
    bool passed = true;
}
```

```

        string name = "Ahmad";

        cout << name << " is " << studentAge << " years old with GPA "
<< gpa << endl;
        return 0;
    }

```

Real-world example: In a student management system, age might be int, gpa a double, and name a string — each type matches the nature of the real-world data it represents.

3. Operators

Arithmetic: + - * / % **Relational:** == != > < >= <= **Logical:** && || !
Assignment: = += -= *= /=

```

int a = 10, b = 3;
cout << a + b << endl; // 13
cout << a % b << endl; // 1 (remainder)
cout << (a > b && b > 0) << endl; // 1 (true)

```

Real-world example: % (modulo) is how you check if a number is even/odd, or how pagination works (itemIndex % itemsPerPage).

4. Control Flow

If-else:

```

int marks = 85;
if (marks >= 90) {
    cout << "Grade A" << endl;
} else if (marks >= 75) {
    cout << "Grade B" << endl;
} else {
    cout << "Grade C" << endl;
}

```

Switch:

```

int day = 3;
switch (day) {
    case 1: cout << "Monday"; break;
    case 2: cout << "Tuesday"; break;
    case 3: cout << "Wednesday"; break;
    default: cout << "Invalid day";
}

```

Loops:

```

// for loop – used when repetitions are known
for (int i = 1; i <= 5; i++) {
    cout << i << " ";
}

// while loop – used when repetitions depend on a condition
int count = 0;
while (count < 5) {
    cout << count << " ";
    count++;
}

```

```
// do-while – runs at least once
int x = 0;
do {
    cout << x << " ";
    x++;
} while (x < 5);
```

Real-world example: A for loop processing every row in an attendance sheet; a while loop retrying a network connection until it succeeds.

5. Functions

Functions let you break a program into reusable, named blocks of logic.

```
#include <iostream>
using namespace std;

// Function declaration
int addNumbers(int a, int b) {
    return a + b;
}

int main() {
    int result = addNumbers(5, 7);
    cout << "Sum: " << result << endl;
    return 0;
}
```

Function overloading (same name, different parameters):

```
int add(int a, int b) { return a + b; }
double add(double a, double b) { return a + b; }
```

Real-world example: A calculateTotal() function used across a billing system so the total calculation logic exists in one place instead of being repeated everywhere.

6. Arrays and Strings

```
int marks[5] = {90, 85, 78, 92, 88};
cout << marks[0] << endl; // 90

for (int i = 0; i < 5; i++) {
    cout << marks[i] << " ";
}
```

Strings:

```
#include <string>
string name = "CodeNotes";
cout << name.length() << endl; // 9
cout << name.substr(0, 4) << endl; // Code
```

Real-world example: Storing a class's exam scores in an array to compute the average, or storing usernames as strings in a login system.

7. Pointers and References

A **pointer** stores the memory address of a variable.

```
int age = 20;
int* ptr = &age; // ptr holds the address of age

cout << age << endl; // 20
cout << ptr << endl; // memory address
cout << *ptr << endl; // 20 (dereferencing – gets the value at
that address)
```

A **reference** is an alias for an existing variable.

```
int score = 50;
int &refScore = score;
refScore = 75; // this also changes score to 75
```

Real-world example: Pointers are how linked lists and trees are built (each node points to the next). References are commonly used to pass large objects to functions without copying them (efficiency).

8. Memory Management

C++ gives you manual control over memory using new and delete.

```
int* ptr = new int; // allocate memory on the heap
*ptr = 100;
cout << *ptr << endl;
delete ptr; // free the memory – prevents memory leaks

int* arr = new int[5]; // dynamic array
delete[] arr;
```

Real-world example: A game that loads a large 3D model into memory needs to delete it when the player exits the level, or the game will slowly consume more and more RAM (a memory leak).

9. Structures

A struct groups related variables under one name.

```
struct Student {
    string name;
    int age;
    double gpa;
};

int main() {
    Student s1;
    s1.name = "Ahmad";
    s1.age = 20;
    s1.gpa = 3.8;
    cout << s1.name << " - GPA: " << s1.gpa << endl;
}
```

Real-world example: Representing a “Point” (x, y) in a graphics program, or an “Employee” record (name, ID, salary) before moving to full classes.

PART 2: OBJECT-ORIENTED PROGRAMMING (OOP)

1. What is OOP?

OOP organizes code around **objects** (real-world entities) instead of just functions and logic. The four pillars are: 1. **Encapsulation** — bundling data and methods, hiding internal details 2. **Inheritance** — reusing code from a parent class 3. **Polymorphism** — one interface, many implementations 4. **Abstraction** — showing only essential features, hiding complexity

Real-world example: A “Car” in real life has properties (color, speed, brand) and behaviors (accelerate, brake). OOP models software the same way.

2. Classes and Objects

A **class** is a blueprint. An **object** is an actual instance created from that blueprint.

```
#include <iostream>
using namespace std;

class Car {
public:
    string brand;
    int speed;

    void accelerate() {
        speed += 10;
        cout << brand << " speed is now " << speed << endl;
    }
};

int main() {
    Car myCar;           // object created from Car class
    myCar.brand = "Toyota";
    myCar.speed = 0;
    myCar.accelerate(); // Toyota speed is now 10
    return 0;
}
```

Real-world example: Car is the blueprint; myCar, yourCar, friendsCar are individual objects — each with their own brand and speed, but sharing the same behavior defined in the class.

3. Constructors and Destructors

A **constructor** runs automatically when an object is created — used to initialize data. A **destructor** runs automatically when an object is destroyed — used for cleanup.

```
class Student {
public:
    string name;

    // Constructor
    Student(string n) {
```

```

        name = n;
        cout << name << " has enrolled." << endl;
    }

    // Destructor
    ~Student() {
        cout << name << " has graduated." << endl;
    }
};

int main() {
    Student s1("Ahmad"); // constructor runs
    // destructor runs automatically when s1 goes out of scope
    return 0;
}

```

Real-world example: When a database connection object is created, the constructor opens the connection; when it's destroyed, the destructor closes it — preventing leaked/hanging connections.

4. Encapsulation

Encapsulation means keeping data **private** and only allowing controlled access through public methods (getters/setters).

```

class BankAccount {
private:
    double balance; // hidden from outside

public:
    BankAccount(double initial) {
        balance = initial;
    }

    void deposit(double amount) {
        if (amount > 0) balance += amount;
    }

    double getBalance() {
        return balance;
    }
};

int main() {
    BankAccount acc(1000);
    acc.deposit(500);
    cout << "Balance: " << acc.getBalance() << endl;
    // acc.balance = -5000; // NOT allowed - balance is private
}

```

Real-world example: You can't directly reach into an ATM's internal balance variable and edit it — you can only deposit/withdraw through defined operations. That's encapsulation protecting the data's integrity.

5. Inheritance

Inheritance lets a class (child/derived) reuse properties and methods of another class (parent/base).

```

class Animal {

```

```

public:
    string name;
    void eat() {
        cout << name << " is eating." << endl;
    }
};

class Dog : public Animal {
public:
    void bark() {
        cout << name << " is barking." << endl;
    }
};

int main() {
    Dog myDog;
    myDog.name = "Rex";
    myDog.eat(); // inherited from Animal
    myDog.bark(); // defined in Dog
}

```

Types of inheritance: single, multiple, multilevel, hierarchical, hybrid.

Real-world example: A “SavingsAccount” and “CurrentAccount” both inherit common behavior (deposit, withdraw) from a base “BankAccount” class, instead of rewriting that logic twice.

6. Polymorphism

Polymorphism means “many forms” — the same function behaves differently depending on the object calling it.

Compile-time polymorphism (function/operator overloading):

```

class Calculator {
public:
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
};

```

Runtime polymorphism (virtual functions):

```

class Shape {
public:
    virtual void draw() {
        cout << "Drawing a shape" << endl;
    }
};

class Circle : public Shape {
public:
    void draw() override {
        cout << "Drawing a circle" << endl;
    }
};

int main() {
    Shape* shape = new Circle();
    shape->draw(); // Output: Drawing a circle (runtime decides
which draw() to call)
    delete shape;
}

```

Real-world example: A ride-sharing app has a generic `calculateFare()` — but Bike, Car, and Premium ride types each calculate fare differently, while the app calls the same function name on all of them.

7. Abstraction

Abstraction hides complex implementation details and exposes only what's necessary, often using **abstract classes** with pure virtual functions.

```
class PaymentMethod {
public:
    virtual void pay(double amount) = 0; // pure virtual function –
    makes this abstract
};

class CreditCard : public PaymentMethod {
public:
    void pay(double amount) override {
        cout << "Paid " << amount << " using Credit Card." << endl;
    }
};

class PayPal : public PaymentMethod {
public:
    void pay(double amount) override {
        cout << "Paid " << amount << " using PayPal." << endl;
    }
};

int main() {
    PaymentMethod* method = new PayPal();
    method->pay(250);
    delete method;
}
```

Real-world example: When you swipe a card at checkout, you don't see (or need to know) how the bank verifies the transaction internally — you just get a "Payment Successful" result. That's abstraction.

8. Static Members

static members belong to the class itself, not to any single object — shared across all instances.

```
class Counter {
public:
    static int count;

    Counter() {
        count++;
    }
};

int Counter::count = 0;

int main() {
    Counter c1, c2, c3;
    cout << "Objects created: " << Counter::count << endl; // 3
}
```

Real-world example: Tracking how many users are currently logged into an app — a static counter increments on login and decrements on logout, shared across the whole system.

9. Friend Functions

A friend function can access private members of a class even though it isn't a member of that class.

```
class Box {
private:
    int length;
public:
    Box(int l) : length(l) {}
    friend void printLength(Box b);
};

void printLength(Box b) {
    cout << "Length: " << b.length << endl; // allowed due to friend
}
```

Real-world example: Used sparingly — e.g., allowing a specific logging/auditing function to inspect private internal state of a class for debugging purposes, without opening that access to everyone.

10. The this Pointer

Inside a class's member functions, this is a pointer to the current object calling the function.

```
class Student {
public:
    string name;
    Student(string name) {
        this->name = name; // 'this->name' is the member, 'name' is
the parameter
    }
};
```

Real-world example: Used constantly when a constructor's parameter name matches the class's member variable name — this->name distinguishes "the object's name" from "the incoming parameter."

Quick Recap Table

Concept	One-line definition
Variable	Named storage for a value
Function	Reusable block of logic
Pointer	Stores a memory address
Class	Blueprint for objects
Object	Instance of a class
Encapsulation	Hiding data, exposing controlled access
Inheritance	Reusing a parent class's code
Polymorphism	Same interface, different behavior

Abstraction Hiding complexity, showing essentials

Notes by Ahmad — CodeNotes